Generalised Langford sequences as a CSP in MiniZinc

Symbolic Explainable AI 2026/27 Group 20: [member names and numbers] **CSPLib problem 024** (Langford's number problem) Video: [video URL]

1 Problem and baseline

Classic Langford L(2,n): place two copies of each $i \in 1..n$ in a sequence of length $2n$ so that the two copies of each i lie $i+1$ positions apart. The baseline, Hakan Kjellerstrand's CSPLib model (`langford2.mzn`, CC-BY-4.0), has $2k$ position variables, one per copy, linked by `position[i+k] = position[i]+i+1`; `all_different` over them; channelling into a `solution` array; symmetry broken on the sequence (`solution[1] < solution[2k]`). It cannot express more copies or other gaps.

2 Extension and model (`langford_gen.mzn`)

Extension. L(s,n) with offset d: s copies of each i, consecutive copies $i+d$ apart, sequence length $N = sn$; $d = 1$ is Langford, $d = 0$ the Nickerson variant. **Viewpoint.** The only decision variables are p_i, the position of the first copy of i. Copy j is at $p_i + j(i+d)$, so the spacing rule holds by construction and the search has n variables instead of sn. Constraints: the last copy fits, $p_i + (s-1)(i+d) \le N$; `all_different` over the sn derived positions; channelling to `seq` (the number at each position), used for output and not searched. **Symmetry.** Reversal maps position q to $N+1-q$, so the reversed first copy of i is $N+1-(s-1)(i+d)-p_i$. A `lex_lesseq` between p and this vector keeps one sequence of each pair. Both are compared in branching order (natural for modes 0/1, largest first for 2/3); lex on `seq` would prefer small numbers first and fight largest-first search. **Parity cut** (optional, $s = 2, d = 1$). Positions $1..2n$ contain n odd ones. An even i has an odd gap, so its copies take one odd and one even position: the even numbers use $\lfloor n/2 \rfloor$ odd positions. An odd i has an even gap, so both its copies have the same parity, and the remaining $m = \lceil n/2 \rceil$ odd positions must be filled by odd numbers in pairs. So m must be even, i.e. $n \equiv 0, 3 \pmod 4$, and then exactly $m/2$ odd numbers start at an odd position. The model posts this count, or `false` when m is odd, in which case MiniZinc rejects the instance during compilation. **Search.** Four modes cross variable order (natural or largest first) with selection (`input_order` or `first_fail`), all `indomain_min`. A redundant `global_cardinality` on `seq` is optional.

3 Explainer (`proj.py`)

For an instance with no solution, `proj.py` prints a readable reason from two necessary conditions. (1) Span: the copies of n span $(s-1)(n+d)+1$ positions, which can exceed N. (2) Sum of positions: all positions sum to $N(N+1)/2 = \sum_i [s\,p_i + \frac{s(s-1)}{2}(i+d)]$, so $\sum_i p_i = n[2(sn+1) - (s-1)(n+1+2d)]/4$ must be an integer; for $s = 2, d = 1$ this is the $n \equiv 0, 3 \pmod 4$ rule. Over the self-test range, 40 of the 68 impossible instances get a reason, including every classic case; the other 28 (18 with $s = 3$, 9 with $s = 4$, one with $s = 2, d = 2, n = 4$) are reported as needing the solver. *Limitation:* the explanation is printed only after the solver returns UNSATISFIABLE, so $n = 14$ without `--parity` times out with no explanation, although the explainer rules it out at once. With `--parity` an equivalent condition is part of the model, which is why those instances are proved immediately.

4 Evaluation

MiniZinc 2.10.1, Gecode 6.4.0, Chuffed 0.14.0; $s = 2, d = 1$; median wall time of 3 runs, 60 s timeout (timeouts not repeated). Every returned solution passes an independent checker; the counts for $n = 7, 8, 11, 12$ (26, 150, 17 792, 108 144, up to reversal) are verified by a brute-force enumerator and consistent with OEIS A014552. `--selftest` compares every model, mode and option on Gecode and Chuffed with the enumerator for $s \in \{2, 3, 4\}$, $d \in \{0, 1, 2\}$, $n \le 10$ ($n \le 9$ for $s = 4$): 1084 solver runs, no mismatches.

Configuration	All sol., $n = 12$		UNSAT	UNSAT
	time (s)	nodes	$n = 13$ (s)	$n = 14$ (s)
Baseline (Gecode)	9.68	1 540 501	51.82	> 60
Mode 1 (natural order)	11.76	1 811 825	> 60	> 60
Mode 3 (largest first)	8.44	1 230 971	47.44	> 60
Mode 3, no sym. breaking	14.77	2 460 807	> 60	> 60
Mode 3 + cardinality	11.47	1 230 971	> 60	> 60
Mode 3 + parity cut	9.55	1 221 331	**0.06**	**0.06**
Mode 3, Chuffed	43.80	388 429	> 60	not run

The parity cut proves $n = 13, 14$ unsatisfiable in 0.06 s without search, where the others need 47–52 s or time out; on solvable instances it removes under 1% of nodes and saves no time. Without symmetry breaking, solutions (216 288) and nodes both double. Mode 3 counts slightly faster than the baseline (8.44 vs. 9.68 s) but mode 1 is slower (11.76 s), so the search modes give no clear gain; modes 0 and 2 were only checked for correctness in the self-test, not benchmarked. The cardinality constraint leaves nodes unchanged and adds time, since `all_different` and channelling already imply it. Chuffed explores fewer nodes but is about five times slower. For a first solution at $n = 15$–24 every configuration, baseline included, takes about 0.08 s, which is start-up time, so no difference can be claimed there.

5 Running the code

Python 3.11.2; MiniZinc 2.10.1 (with Gecode 6.4.0 and Chuffed 0.14.0) on the `PATH`; `pip install minizinc==0.10.0`; both `.mzn` files next to `proj.py`. The command line runs only the generalised model (options `-s -d --solver --mode 0..3 --all --no-sym --gcc --parity --timeout`); the baseline runs inside `--benchmark` and `--selftest`.

```
python3 proj.py 8                       # L(2,8), first solution
python3 proj.py 7 -s 3 -d 2 --all       # all solutions of L(3,7) with d=2
python3 proj.py 14 --parity             # UNSAT, with explanation
python3 proj.py --selftest              # all models and options vs. brute force
python3 proj.py --benchmark --reps 3 --timeout 60 --out results.csv
```

6 Use of generative AI and web resources

I used Claude Code (Anthropic) during development. It diagnosed that the code did not run because of an old MiniZinc installation and set up MiniZinc 2.10.1. It found that my original symmetry-breaking constraint, a comparison on the sequence, conflicted with the search order, and rewrote it as a comparison on the position variables. In `proj.py` it wrote the brute-force enumerator and the self-test, the solver selection, and changes that make the benchmark robust. It made two small edits to the baseline file so that it takes its size from `proj.py`, and it ran the benchmark and interpreted the results. It also supplied the reference counts from OEIS A014552, which I confirmed with the enumerator. The generalisation, the parity argument, the search modes and the explainer are my own design. Claude, in the Claude app, helped structure and write this report from my code and results, and I checked it. Web sources [how these were used]: dialectrix.com/langford.html, dialectrix.com/langford/langford-biblio.html, MathWorld, "Langford's Problem", lclark.edu/~miller/langford (Internet Archive, 1999).